

Отчёт о работе над проектом

«Интеллектуальная система планирования доходов»

Сергей Рачков

Роль: team lead, backend-разработчик

Июль 2026

Содержание

1	Введение	2
2	Архитектурные решения	2
2.1	Выбор технологического стека	2
2.2	Ключевые архитектурные решения	2
3	Выполненные работы	3
3.1	Backend (Java Spring Boot)	3
3.2	Python-микросервис (FastAPI)	3
3.3	Фронтенд (React + TypeScript)	3
3.4	Тимлидство и координация	4
3.5	Работа с GigaChat	4
4	Тестовые данные	4
5	Заключение	4
6	Скриншоты работы продукта	5

1 Введение

Проект «**Интеллектуальная система планирования доходов**» разрабатывался в рамках учебного кейса. Цель - создать MVP сервиса-помощника, который позволяет загружать данные, строить граф бизнес-модели, генерировать сценарии прогнозов, находить аномалии и получать новости по теме.

Моя роль в команде - **team lead и ведущий backend-разработчик**. Я отвечал за архитектуру, структуру проекта, написание бэкенда, интеграцию с Python-микросервисом, координацию работы фронтенда, а также участвовал в разработке Python-сервиса и помогал с генерацией страниц фронтенда.

2 Архитектурные решения

2.1 Выбор технологического стека

- Бэкенд: Java 17, Spring Boot 3.
- База данных: H2 (embedded).
- Фронтенд: React 18 + TypeScript + Vite.
- UI-библиотеки: Tailwind CSS (стилизация), Recharts (графики), React Flow (граф), Zustand (управление состоянием).
- Python-микросервис: FastAPI, statsmodels, Prophet.
- WebSocket

2.2 Ключевые архитектурные решения

- **Длинный (long) формат хранения данных.** Все факты хранятся в одной таблице `facts` с полями: `period`, `subject`, `indicator`, `value`, `unit`, `district`. Это позволяет гибко добавлять новые показатели без изменения схемы БД и упрощает агрегации.
- **JSON-хранение сценариев.** Параметры и результаты сценариев хранятся в полях `paramsJson` и `resultJson` типа CLOB. Это даёт возможность изменять структуру сценария без миграций.
- **Граф бизнес-модели.** Узлы и рёбра хранятся отдельно; рёбра содержат операторы (+, -, *, /, %). Такая структура позволяет визуально редактировать зависимости и вычислять производные показатели.
- **Интеграция с Python.** Тяжёлые вычисления (прогнозирование, анализ новостей) вынесены в отдельный микросервис на FastAPI. Это позволило использовать специализированные ML-библиотеки и не перегружать основной бэкенд.
- **Кэширование новостей на бэкенде** (5 минут) и на фронтенде (`sessionStorage`). Это снижает нагрузку на Python-сервис и ускоряет ответ.
- **Лимитирование параллельных запросов.** На фронтенде реализован лимитер (максимум 5 одновременных тяжёлых операций) – предотвращает перегрузку сервера при массовой генерации сценариев.

3 Выполненные работы

3.1 Backend (Java Spring Boot)

Я разработал следующие основные компоненты:

- **UploadDataService** - загрузка Excel с upsert-логикой (обновление существующих записей, вставка новых).
- **ScenarioService** - генерация сценариев с учётом графа, расчёт доверительных интервалов, вычисление драйверов.
- **GraphService** - управление графом бизнес-модели (CRUD узлов и рёбер).
- **NewsService** - получение новостей через Python (с кэшированием и интеграцией WebSocket).
- **AnomalyService** - детекция аномалий методом IQR с возможностью замены и восстановления.
- **DriversService** - расчёт динамики показателей и их вклада.
- **SeasonalityService** - расчёт сезонных коэффициентов (впоследствии был удалён, так как аналогичная функциональность реализована в Python, но в процессе разработки он был написан).

Также реализованы все REST-эндпоинты, CORS-настройки, WebSocket-конфигурация (для прогресса загрузки и новостей), глобальный обработчик исключений.

3.2 Python-микросервис (FastAPI)

Мной написан основной файл `app.py`, содержащий:

- Эндпоинт `/predict` - вызывает сравнение ML-моделей (SARIMAX, Prophet, Exponential Smoothing, STL, Ridge, Croston) и возвращает прогнозы.
- WebSocket `/ws/news` - асинхронный сбор и анализ новостей с промежуточными статусами для клиента.
- Эндпоинты `/generate-news` и `/generate-text` (последний был удалён в процессе рефакторинга, но первоначально написан).
- Интеграция с GigaChat для анализа новостей и генерации резюме.
- Парсинг RSS-лент через `news_parser.py` (в написании которого я участвовал совместно с другими коллегами).

Модуль `ai_prediction.py` был написан аналитиком самостоятельно, я лишь интегрировал его в сервис.

3.3 Фронтенд (React + TypeScript)

Моё участие во фронтенде заключалось в:

- Согласовании общей структуры страниц и дизайна (совместно с командой).
- Активной помощи в разработке страницы «Сценарии» - практически полностью сгенерировал компоненты `ScenarioDetailView` и `ScenarioForm` (с помощью AI, но с полным контролем логики).
- Участии в создании компонентов для графа (инспектор узлов, авто-лэйаут).

- Рефакторинге и удалении устаревших частей (например, моков, дублирующихся компонентов).
- Исправлении багов в визуализации графиков (например, смещение подписи «Прогноз»).

Таким образом, я обеспечил связность между бэкендом и фронтендом, а также следил за целостностью пользовательского опыта.

3.4 Тимлидство и координация

Как team lead, я выполнял следующие функции:

- Определение общей структуры репозитория и модулей.
- Контроль над архитектурной целостностью (чтобы изменения не нарушали уже работающие части).
- Распределение задач между участниками команды.
- Проведение код-ревью и помощь коллегам в сложных участках.
- Рефакторинг и устранение легаси (например, объединение дублирующих эндпоинтов, удаление неиспользуемых сервисов, приведение документации в порядок).

3.5 Работа с GigaChat

Интеграция с GigaChat была особенно сложной: модель часто отказывалась давать ответы в нужном формате (JSON). Совместно с коллегами мы доработали промпты, добавили логику повторных запросов при отказе и фильтрацию «острых» тем. В результате новостной пайплайн стал стабильным и даёт полезные аналитические метки.

4 Тестовые данные

В процессе разработки использовались тестовые Excel-файлы, которые также включены в репозиторий:

- `canteen_bananas.xlsx` - данные о продажах бананов по регионам за несколько лет.
- `core_test_database.xlsx` - расширенный набор данных для проверки всех функциональностей.
- `table_from_original_case.xlsx` - исходные данные кейса (доход банка, студенты, столовая).

Эти файлы служили для валидации расчётов и демонстрации.

5 Заключение

За время работы над проектом я:

- разработал и реализовал архитектуру бэкенда;
- написал большую часть Java-сервисов и интегрировал их с Python;
- настроил WebSocket, REST API, CORS и обработку ошибок;
- координировал работу команды, помогал с фронтендом и исправлял ошибки коллег;

- участвовал в создании Python-сервиса и настройке GigaChat;
- привёл документацию в соответствие с текущим состоянием кода и удалил легаси.

В результате получился работающий MVP, демонстрирующий все заявленные функции: загрузку данных, граф, сценарии, аномалии, новости и экспорт. Опыт работы с GigaChat и интеграцией ML-моделей был особенно ценным.

6 Скриншоты работы продукта

Ниже приведены скриншоты ключевых экранов, демонстрирующие функциональность разработанного сервиса. Все скриншоты сделаны в процессе тестирования.

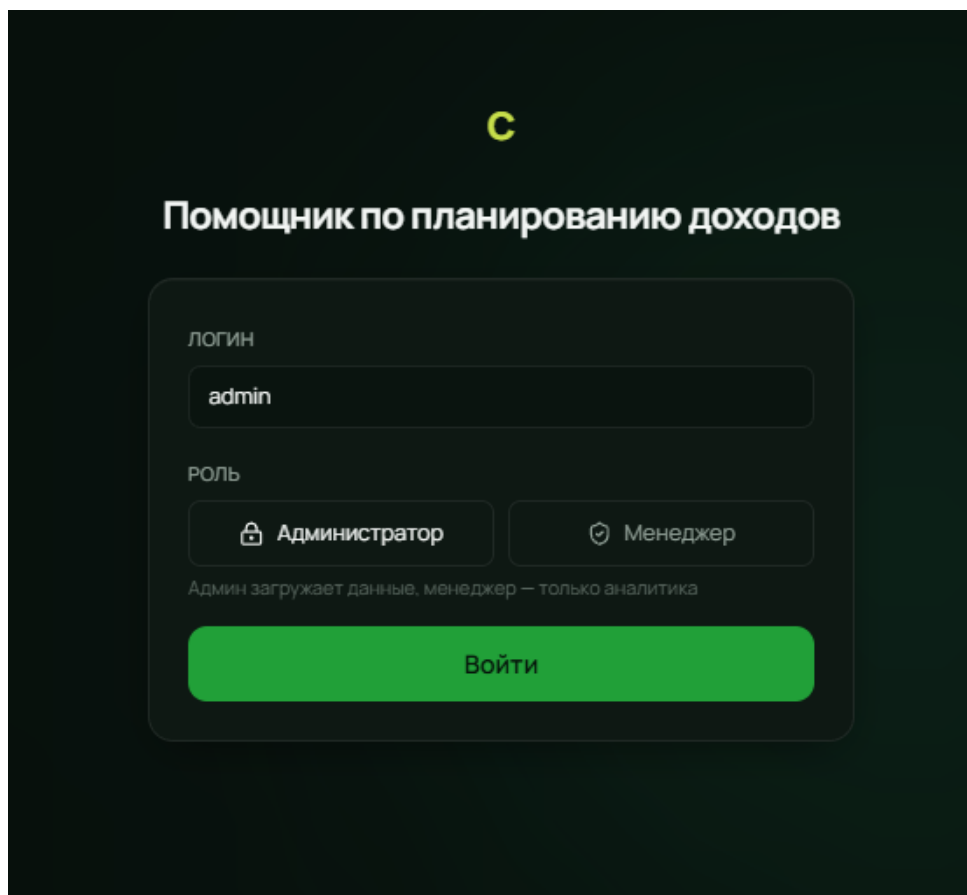


Рис. 1: Экран входа

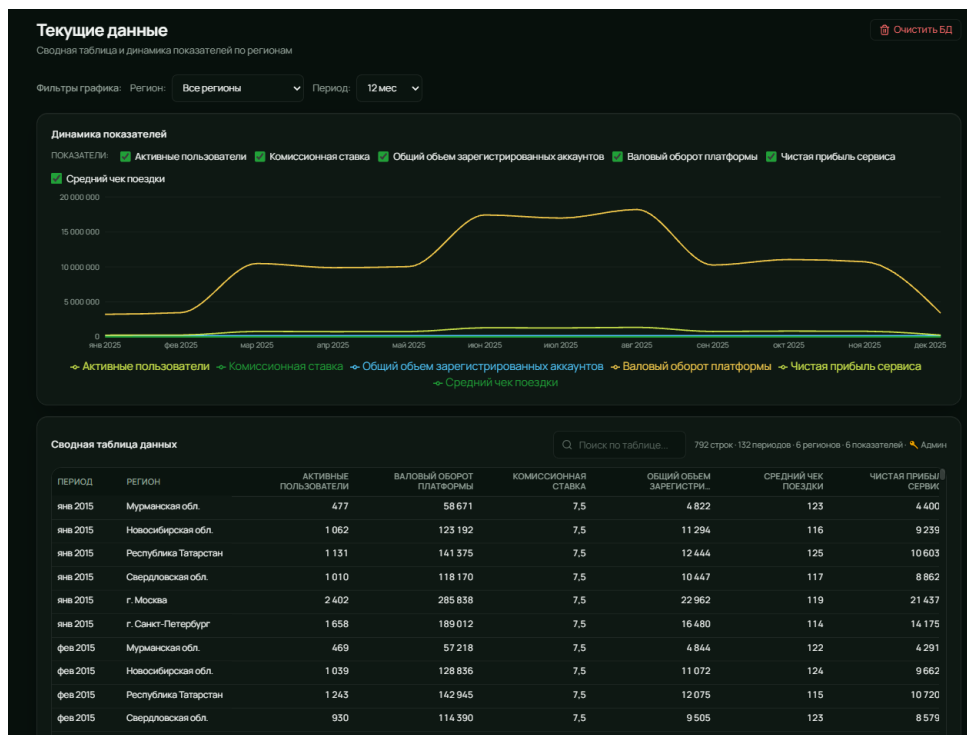


Рис. 2: Текущие данные – сводная таблица и график

```
D:\Work\JavaPr\SPERBANK_practice26\Bank-tables-AI-analityc\python-service>python app.py
Importing plotly failed, Interactive plots will not work.
INFO: Will watch for changes in these directories: ['D:\Work\JavaPr\SPERBANK_practice26\Bank-tables-AI-analityc\python-service']
INFO: Uvicorn running on http://127.0.0.1:5000 (Press CTRL+C to quit)
INFO: Started reloader process [32628] using WatchFiles
Importing plotly failed, Interactive plots will not work.
INFO: Started server process [36256]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

```
Выйти C:\Windows\system32\cmd.exe

VITE v5.4.21 ready in 1106 ms

  Local:   http://localhost:5173/
  Network: http://26.68.87.48:5173/
  Network: http://192.168.10.22:5173/
  Network: http://192.168.56.1:5173/
  Network: http://169.254.217.165:5173/
  press h + enter to show help
```

```
Terminal Local (2) x

2026-07-17 20:33:45 - HHH000026: Second-level cache disabled
2026-07-17 20:33:45 - No LoadTimeWeaver setup: ignoring JPA class transformer
2026-07-17 20:33:46 - HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA plat
2026-07-17 20:33:46 - Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-07-17 20:33:46 - Hibernate is in classpath; If applicable, HQL parser will be used.
2026-07-17 20:33:47 - spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed durin
configure spring.jpa.open-in-view to disable this warning
2026-07-17 20:33:47 - Tomcat started on port 8080 (http) with context path '/'
2026-07-17 20:33:47 - Starting...
2026-07-17 20:33:47 - BrokerAvailabilityEvent[available=true, SimpleBrokerMessageHandler [org.springframework.messaging
tionRegistry@25f688d5]]
2026-07-17 20:33:47 - Started.
2026-07-17 20:33:47 - Started UniversityAnalyticsApplication in 3.244 seconds (process running for 3.529)
<=====> 80% EXECUTING [24s]
> :bootRun
[ ]
```

Рис. 3: Запущенные консоли (бэкенд, Python, фронтенд)

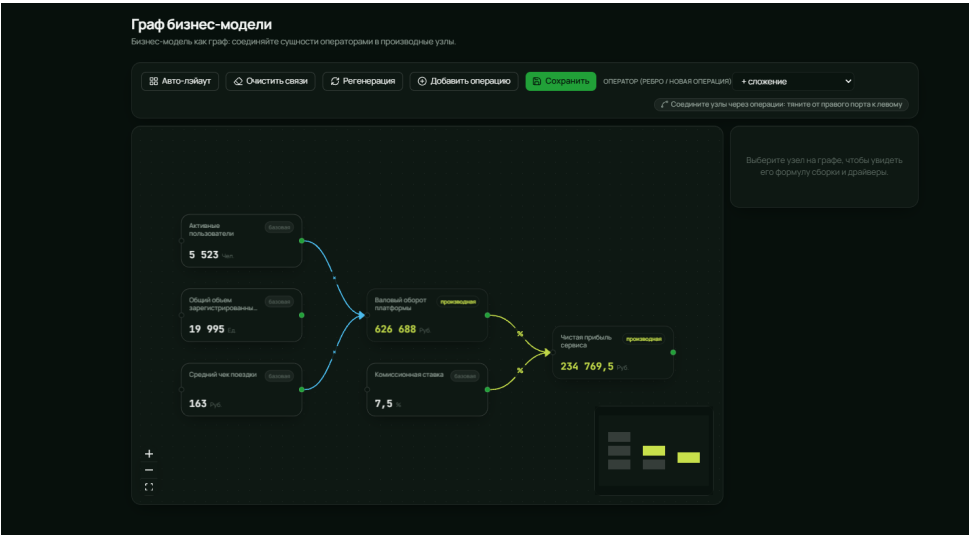


Рис. 4: Граф бизнес-модели

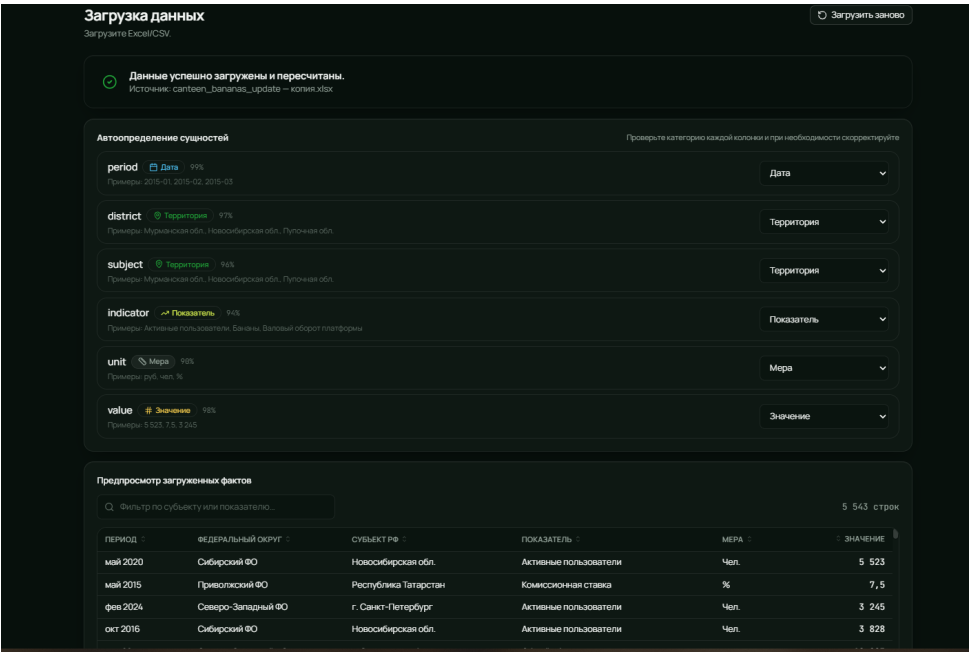


Рис. 5: Загрузка данных – предпросмотр и сущности

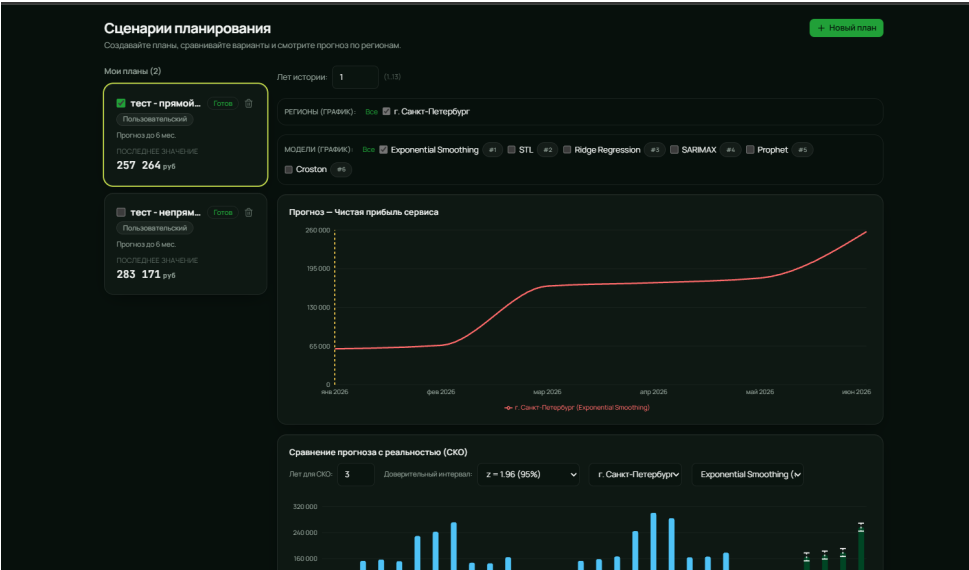


Рис. 6: Сценарии – прогноз с настройками

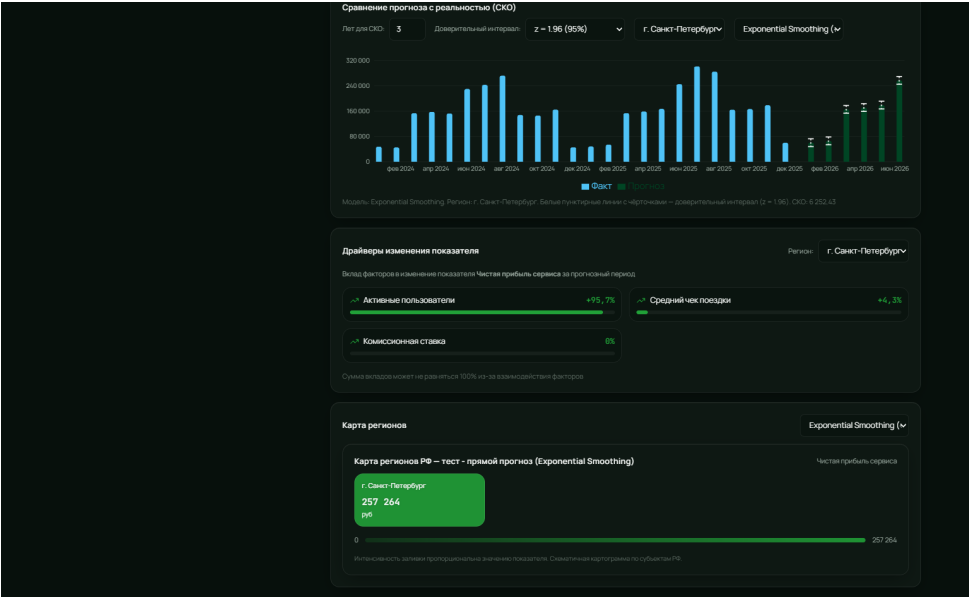


Рис. 7: Сценарии – карта регионов

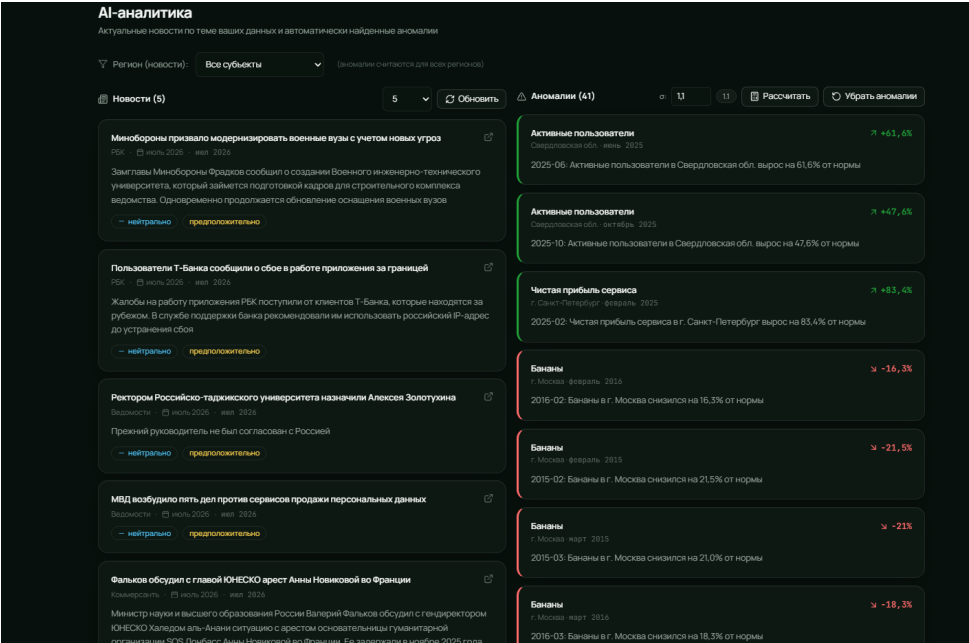


Рис. 8: AI-аналитика: новости и аномалии

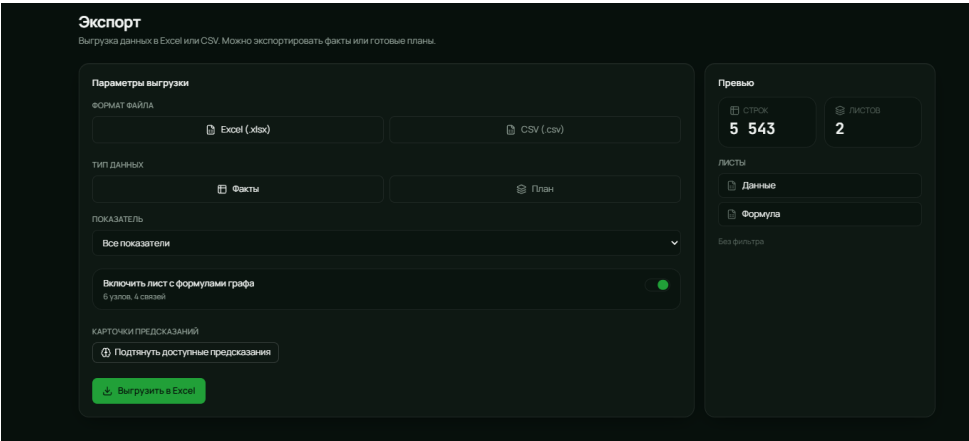


Рис. 9: Экспорт данных

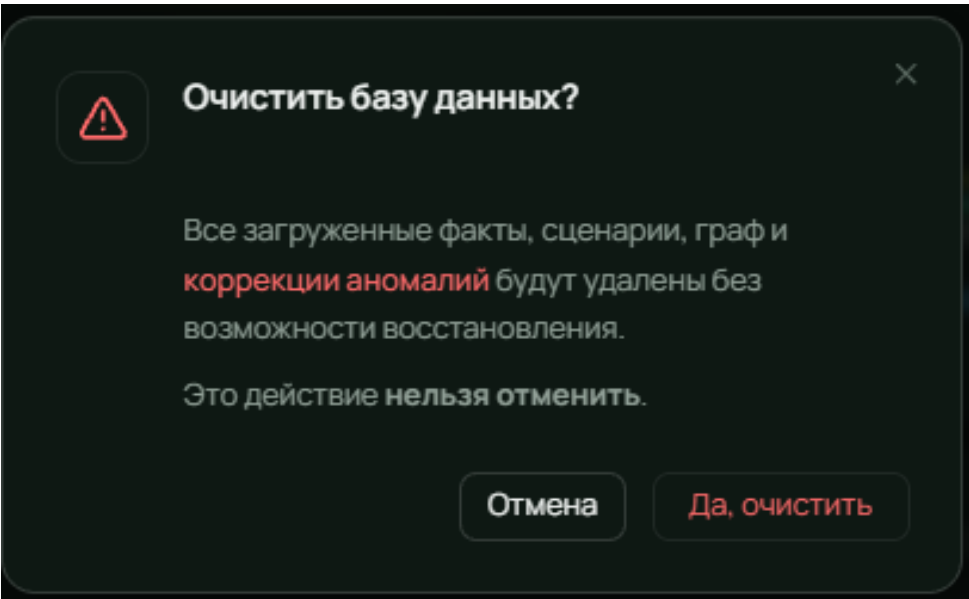


Рис. 10: Очистка базы данных (администратор)